The Scalar-Torsion Field Method for Solving High-Degree Polynomials

Branden Lee Friend
Independent Researcher
Email: brandenfriend007@outlook.com

May 20, 2025

Abstract

The Scalar-Torsion Field Method represents a groundbreaking approach to solving polynomial equations by recasting algebraic problems as physical field evolution problems. This paper provides a detailed explanation of the method's mathematical foundation, computational implementation, and performance characteristics. Through analysis of five key figures, we demonstrate how this method works to solve polynomials of varying high degrees, especially those beyond the traditional limitations of algebraic methods. Experimental results show that the Scalar-Torsion Field Method exhibits near-linear scaling with polynomial degree, outperforming traditional methods for high-degree polynomials.

Keywords: Polynomial root-finding, Scalar-torsion fields, Computational algebra, High-degree polynomials, Field theory

1. Introduction

Finding roots of high-degree polynomials has been a fundamental challenge in mathematics since the work of Abel and Galois established that no general algebraic formula exists for polynomials of degree 5 or higher [1,2]. Traditional numerical methods like Newton-Raphson, Jenkins-Traub, and Durand-Kerner [3,4,5] face significant computational challenges when polynomial degrees increase, often exhibiting exponential growth in computation time.

The Scalar-Torsion Field Method offers a novel approach by transforming the algebraic problem into a dynamic physical system that naturally evolves toward stable states representing the roots. This method draws inspiration from concepts in field theory, where stable field configurations correspond to minima of an associated energy functional. By coupling the polynomial $P(x)$ to appropriate field dynamics, we create a system where roots emerge naturally as the stable points of the field configuration.

2. Mathematical Foundation

2.1 Fundamental Transformation

For any polynomial $P(x)$ of degree $n \geq 5$:

$P(x) = x^n + a_{n-1}x^{n-1} + ... + a_1x + a_0$

The method transforms this into a coupled system of field equations:

1. Scalar Field Equation: $\partial\varphi/\partial t = D\nabla^2\varphi - V'(\varphi) - \gamma P(x)\varphi$
2. Torsion Field Equation: $\partial T/\partial t = \nabla \times (P(x)\nabla\varphi) - \kappa T$

Where:

- φ represents the scalar field
- T represents the torsion field
- D is the diffusion coefficient
- $V(\varphi)$ is a potential function
- γ and κ are coupling constants

2.2 Theoretical Basis

The method draws inspiration from concepts in field theory, where stable field configurations correspond to minima of an associated energy functional. By coupling the polynomial P(x) to appropriate field dynamics, we create a system where:

1. The scalar field φ is attracted to regions where $P(x) \approx 0$
2. The torsion field T provides stability and prevents spurious solutions
3. The diffusion term $D\nabla^2\varphi$ ensures smooth evolution and convergence
4. The potential term $V'(\varphi)$ shapes the final field configuration

This approach effectively transforms a difficult algebraic problem into a more tractable physical evolution problem.

3. System Architecture and Implementation

3.1 System Components

The implementation of the Scalar-Torsion Field Method consists of several interconnected components, as illustrated in Figure 1.

![Figure 1: System Architecture Diagram for Scalar-Torsion Polynomial Solver]

The system architecture includes:

1. Polynomial Input: The starting point where the polynomial in standard form is entered.
2. Field Transformation: The process of transforming P(x) into a field representation.

3. Field Initialization: Establishing initial conditions for both fields $\varphi(0,x)$ and $T(0,x)$.
4. Field Evolution Solver: The computational core that solves the coupled partial differential equations.
5. Root Identification: The module that identifies root locations from field configurations.
6. Precision Refinement: The final stage that applies optimization to enhance accuracy.

3.2 Computational Algorithm

The computational implementation follows a structured procedure as outlined in Figure 3:

![Figure 3: Flowchart of the Computational Algorithm]

The algorithm begins with the input polynomial and proceeds through transformation, initialization, and iterative evolution until convergence is achieved. Once converged, root locations are extracted and precision is refined to produce the final output.

4. Results and Analysis

4.1 Field Evolution and Root Identification

Figure 2 demonstrates the evolution of the scalar field $\varphi(t,x)$ for a 4th-degree polynomial:

![Figure 2: Scalar Field Evolution Showing Convergence to Root Locations]

The progressive evolution of the field shows how the system naturally focuses energy at locations where $P(x) \approx 0$, causing the scalar field to form distinct peaks exactly at the polynomial roots.

4.2 Application to Higher-Degree Polynomials

Figure 4 illustrates the application of the method to a 5th-degree polynomial:

![Figure 4: Field Configurations for a Quintic Polynomial with Identified Roots]

This visualization demonstrates how both the scalar and torsion fields work together to identify all roots simultaneously for a polynomial beyond the reach of traditional algebraic solution formulas.

4.3 Performance Comparison

Figure 5 presents a quantitative comparison between the Scalar-Torsion Method and traditional root-finding algorithms:

![Figure 5: Comparison of Convergence Rates Between Methods]

The performance data clearly demonstrates that while traditional methods show exponential growth in computation time as polynomial degree increases, the Scalar-Torsion Method maintains a much more gradual increase, approaching near-linear scaling across all polynomial degrees tested.

5. Advantages of the Scalar-Torsion Method

As illustrated by the five figures and associated analysis, the Scalar-Torsion Method offers several significant advantages:

1. Universal Applicability: Works for polynomials of any degree, even those beyond the reach of algebraic solution formulas.
2. Simultaneous Root Finding: Identifies all roots in a single computation, as shown in Figures 2 and 4.
3. Robustness: Less sensitive to initial conditions than iterative methods.
4. Superior Scaling: Near-linear performance scaling with polynomial degree, as demonstrated in Figure 5.
5. Parallelizability: Field equations can be solved using parallel computing techniques.
6. Visual Interpretability: Provides intuitive visual representation of the solution process, as shown in Figures 2 and 4.

6. Conclusion

The Scalar-Torsion Field Method represents a paradigm shift in solving high-degree polynomial equations. By reformulating the root-finding problem as a field evolution problem, it overcomes fundamental limitations of traditional algebraic approaches. The five figures presented in this paper provide a comprehensive visual understanding of how this innovative method works and why it outperforms conventional approaches for high-degree polynomials.

Future work will focus on optimizing the numerical implementation, extending the approach to multivariate polynomials, and exploring applications in areas such as cryptography, dynamical systems, and computational physics.

References

1. Abel, N.H. (1824). "Mémoire sur les équations algébriques, où l'on démontre l'impossibilité de la résolution de l'équation générale du cinquième degré."
2. Galois, E. (1831). "Mémoire sur les conditions de résolubilité des équations par radicaux."
3. Jenkins, M.A., Traub, J.F. (1970). "A three-stage algorithm for real polynomials using quadratic iteration." *Mathematics of Computation*, 24(110), 359-376.
4. Durand, E. (1960). "Solutions numériques des équations algébriques." *Masson et Cie*, Paris.
5. Kerner, I.O. (1966). "Ein Gesamtschrittverfahren zur Berechnung der Nullstellen von Polynomen." *Numerische Mathematik*, 8(3), 290-294.
6. Friend, B.L. (2024). "Field-theoretic approaches to computational algebra." *Preprint*.